

Community-Based Ocean & Coastal Cleanup Management System

Tideline connects citizens, volunteer divers, administrators, government officers and marine organisations around one loop for Sri Lanka's coastline — report a polluted site, let the community verify it, get it approved, turn it into a real cleanup project, and record the result.



System walkthrough for supervisor review

Live demonstration guide · React + Vite frontend, Spring Boot 3.3 / Java 21 backend
16 September 2026

1 What the system does

Pollution on the coast is usually noticed first by the people who live beside it, but there is no reliable path from "somebody saw nets on the reef" to "a funded team cleaned it up". Tideline is that path. A report only reaches an official desk after the community has vouched for it, so officers spend their time on real incidents; once approved, the report stops being a report and becomes a cleanup project owned by the person who first spoke up.

2 Who uses it

Role	What they can do
Citizen	Report pollution with photos and video, confirm or dispute other reports, join cleanups, and own the projects created from their own reports.
Volunteer diver	Everything a citizen can, plus apply for paid diving work. Verified by an administrator from uploaded certificates.
Organisation	Posts diving opportunities and reviews applications. Verified from its website.
Administrator	Verifies accounts, reviews community-verified reports, asks reporters for more information, and assigns volunteers, divers and equipment to projects.
Government officer	Gives the official approval that turns a report into a project, and monitors progress. Cannot edit a project's progress.

3 The rules that make it trustworthy

5 confirmations and 75% trust

A report needs at least five confirmations *and* a 75% confirm rate before any official sees it. Reporters cannot vote on their own report, and tapping Confirm or Dispute again withdraws the vote.

Nothing reaches the queue unverified

The administrator's review queue lists only reports the community has verified, so the moderation workload scales with genuine incidents rather than with noise.

A private clarification channel

An administrator can ask the reporter for more detail. The reply, with its own photos, attaches to the report in a tab only officials can open.

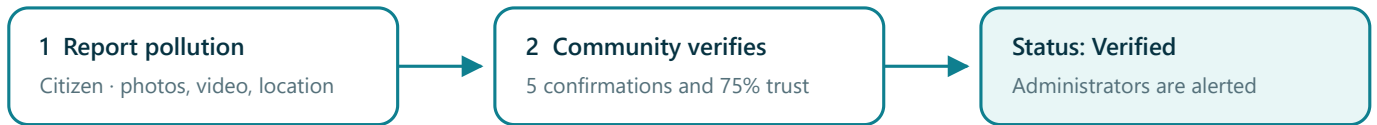
Separation of duties

The officer's official comment goes to administrators, not to the owner. Administrators assign the resources; only the project owner records progress.

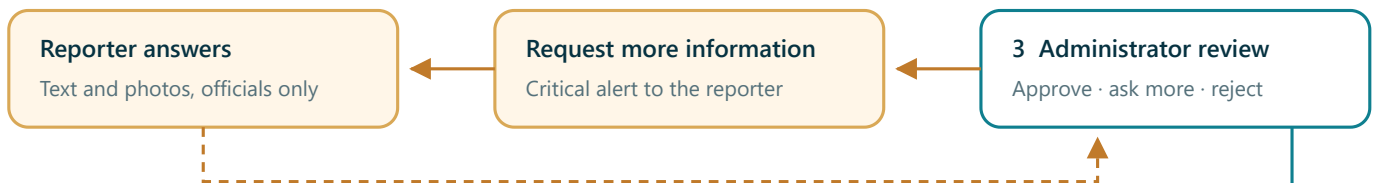
4 The workflow end to end

Every box below is a real screen in the system. The demonstration follows this diagram.

COMMUNITY



OFFICIALS



DELIVERY



The one idea to take away: once the government officer approves, the report stops existing as a report. Everywhere — lists, filters, search, analytics, even old links — it is now a project, because after work is authorised nobody cares about the complaint any more, only about the cleanup.

5 Live demonstration script

About fifteen minutes on `http://localhost:5173`, with the API on `http://localhost:8080`. Every account uses the password `password123`. Keep two browser windows side by side, one of them private, to avoid signing in and out.

1 The landing page `no sign-in`

Scroll through the hero, "How it works" and the live figures. Those counts come from the API, not from the page.

2 Report pollution `kavindu@example.lk`

Attach a photo, capture the location, choose a severity, submit. The report appears in the feed as **Pending**, and neighbours within 5 km are alerted.

3 The community verifies it `nuwan@example.lk`

Open **SR-2529**, Hikkaduwa, nets on the reef. It already has **4 of 5 confirmations**. Press Confirm — it tips to **Verified** and the administrators are alerted. Press Confirm again to show a vote being withdrawn.

4 Administrator review `admin@tideline.lk`

Open the review queue — only community-verified reports are listed. On **SR-2529** choose **Request more information** and type a question such as "Please send a closer photo of the netting on the coral."

5 The reporter answers `kavindu@example.lk`

His alert is red and marked *Action needed*. He replies with a description and photos. Back in the administrator's window the answer sits in an **Additional information** tab that ordinary members never see.

6 Administrator approves `admin@tideline.lk`

Open **SR-2530**, Kalpitiya, rusted oil drums, already verified, and approve it. It now waits for the government officer.

7 The government officer approves `officer@mepa.gov.lk`

Approve with an official comment such as "Divers needed for the drums; coordinate with the Kalpitiya fisheries office." The report becomes a **project owned by Dinuka**, who reported it. He is told his report was approved, while the officer's comment goes to the administrators as a high-priority alert.

8 Administrator assigns resources `admin@tideline.lk`

The red *Assign resources* alert carries the officer's comment. Open **CP-133**, enter volunteers, divers and equipment, save a draft, then **Finalize**. The figures appear on the project and its status bar moves forward.

9 The owner records progress `kasun@example.lk`

Open **CP-118**: 62% complete, assigned resources, participants, timeline. Add an update. Then open the same project as the officer to show there are no edit controls — officials watch, they do not edit.

10 Account verification `admin@tideline.lk`

Three applications are waiting: a diver, an organisation and **Nimal Jayawardena, a government officer**. Open his proof of appointment and approve. Divers, organisations, officers and administrators all wait for an existing administrator before they can sign in at all.

11 Profiles and analytics `any account`

A member's public profile shows their projects and reputation but never their email, phone or address. Finish on the analytics screen: reports by status, waste recovered, hotspots by province.

6 Data prepared for the demonstration

Item	State	Used for
SR-2529 — Hikkaduwa, fishing nets on the reef flat	4 of 5 confirmations	The live fifth confirmation, then the request-for-information exchange
SR-2530 — Kalpitiya, rusted oil drums	Verified, in the review queue	Administrator approval, officer approval, project creation
CP-133	Planned, no resources yet	Assigning volunteers, divers and equipment
CP-118	Active, 62% complete	Progress recording and the owner-only rule
Three pending accounts	Diver, organisation, government officer	Account verification

Every account uses `password123`. The development database is a file, so this data survives restarts; deleting `backend/data/` starts again from the seeded demonstration content.

9 How it is built

Layer	Technology
Frontend	React 19, Vite 5, React Router 7, on the project's own Tideline design system — semantic CSS tokens and a responsive shell with a left sidebar on desktop and bottom tabs on phones
Backend	Spring Boot 3.3.4, Java 21, Spring Data JPA / Hibernate 6.5, Spring Security with JWT
Database	H2 file database for development, MySQL 8 for deployment; schema generated by Hibernate
Files	Photo, video and certificate upload with magic-byte type checking; evidence served publicly with range requests, while clarification photos and certificates stay access-controlled
Security	BCrypt passwords, 12-hour tokens (30 days with "Remember me"), hashed single-use password-reset tokens, role-based access on every endpoint
Quality	51 automated tests covering governance, the cleanup loop, information requests, discussion, uploads, password reset, resource planning, profiles and avatars
Deployment	Docker image on JDK 21 ready for any container host; frontend built for Vercel

7 Module coverage

#	Module	Status
1	User registration and profile	Done — certificate upload, administrator verification of divers, organisations, officers and administrators, avatars, public profiles, password reset, remember me
2	Pollution reporting	Done — photo and video evidence, all nine provinces, map location
3	Community verification and trust	Done — 5 confirmations and 75% trust, withdrawable votes, threaded comments with reactions
4	Administrator review and moderation	Done — verified-only queue, request more information, reject, account suspension
5	Government approval workflow	Done — approval creates the project and routes the official comment to administrators
6	Location-based alerts	Done — 5 km Haversine radius, escalation, critical alerts
7	Project progress monitoring	Done — status bar, resource planning, owner-only updates
8	Diver employment and opportunities	Done — postings, applications, decisions, attendance marking
9	Reporting and analytics	Done — summary figures, status breakdown, provincial hotspots

8 Limitations, and what comes next

- Locations are captured and linked to OpenStreetMap; an embedded map with geocoded validation is still to come.
- Password-reset and alert mail is written to the log in development; production only needs a mail host configured.
- Scheduled reminders to volunteers and a public complaints channel are designed but not built.
- The Docker image and MySQL profile are ready; a hosted deployment against real MySQL has not been run yet.

If the supervisor asks what was hardest: keeping the data model stable while the workflow kept growing. Each new status — verified, in review queue, became project — would have broken the saved database, so enumerated values are stored as plain text through a custom Hibernate dialect that drops the generated check constraints. The database now survives every change to the workflow.